Handoff Note

This Alteryx pipeline pulls setlists from setlist.fm's API and album tracklists from MusicBrainz API, then models them in Snowflake, then gets connected to two Tableau dashboards.

How to refresh

1. **Pull setlists:** run `Music API Workflow.yxmd.` Register for and save a setlist.fm key in a local key file it reads, and do not paste it into the workflow. Output: `unclean_setlist_data.csv`.
2. **Pull albums and tracks:** run `MusicBrainz API Workflow.yxmd`. You can adjust the user-agent header to your own scenario. Make sure the throttles are very low so it doesn't return a 503 error. Outputs: the release-groups and tracks CSVs.
3. **Load raw tables:** in Snowflake, load `DS_SETLIST_DATA_RAW`, `DS_RELEASE_GROUPS_RAW` and `DS_TRACKS_RAW` by creating a new table with each.
4. **Rebuild the model:** run the DDL/DML script top to bottom (`DS_ARTIST` → `DS_VENUE` → `DS_ALBUM` → `DS_SONG` → `DS_SHOW` → `DS_SONG_PERFORMANCE`), then the check queries at the end.
5. **Refresh Tableau:** refresh the Snowflake connection and republish both dashboards to the server.

What to watch out for

- **Rate limits:** setlist.fm allows about 2 calls/second and 1,440/day (the workflow throttles to 60/minute and retries 429s). MusicBrainz allows 1/second and reports "server busy" quite a lot even if the throttle is set to less than 1/second, so make sure the throttle rate is quite low (around 2-6 records/min).
- **User-logged data:** recent shows may not be logged yet. Joint tours sit under separate artist entries: Kendrick's 2025 Grand National Tour with SZA is missing, which affects the days to road numbers.
- **Not every setlist is a concert:** TV tapings, pop-ups and festival sets are logged too. I used a tour-show rule (tour name exists and 15+ songs) wherever only full shows should count.
- **Song matching:** titles are matched after a normalization that happens with the song key, and about 85% of songs played match a core album. setlist.fm labels collaborations and side projects as covers, so some songs labeled as Covers may not actually be so.

What I'd build next

- **Automate the load:** write Alteryx output straight to Snowflake and schedule a weekly run instead of manually uploading.

- **Add demand and financial data:** ticket prices (maybe through Ticketmaster Discovery API) and listener counts (using Last.fm API) to show whether new-album shows sell as well as catalogue-heavy ones.
- **Routing map:** Show a map using MAKELINE functions that draws out an artists tour to see where more efficient routing is possible.
- **Grow the roster:** adding an artist only needs their MBID.

Artist Analytics – Data Dictionary & Schema

Two APIs (setlist.fm and MusicBrainz) are fed into a Snowflake star schema with two fact tables (shows, songs played at shows) and four dimension tables, joined on MusicBrainz artist ids and a normalized song title. The MusicBrainz ID is a shared ID that is used on MusicBrainz and on the setlist.fm API, so this is a simple join.

Sources

Both sources are pulled with the Alteryx Download tool and joined on the MusicBrainz artist id (MBID), which setlist.fm uses natively.

Source	Method	What it provides	Raw table	Raw grain	Rows
setlist.fm API	`/artist/{mbid}/setlists`, paged by `total` (20 per page), throttled to 1 call/sec, 429 retries	Every show for 5 artists: date, venue, tour, songs in order	`DS_SETLIST_DATA_RAW`	One row per setlist (show)	3,448
MusicBrainz API	`/release-group?artist={mbid}&type=album`	Every album-type release per artist	`DS_RELEASE_GROUPS_RAW`	One row per release group (album)	104

MusicBrainz API	`/release?release-group={id}&inc=recordings`, flattened to tracks in Alteryx; busy-server calls backfilled and unioned	Tracklists of every edition of each album	`DS_TRACKS_RAW`	One row per track per release edition	5,745

Artists I chose: Bruno Mars, Mac DeMarco, Kendrick Lamar, Sabrina Carpenter, Daniel Caesar. The API key is read from a local file, not stored in the workflow. To make your own key, go to https://www.setlist.fm/settings/apps.

Tables

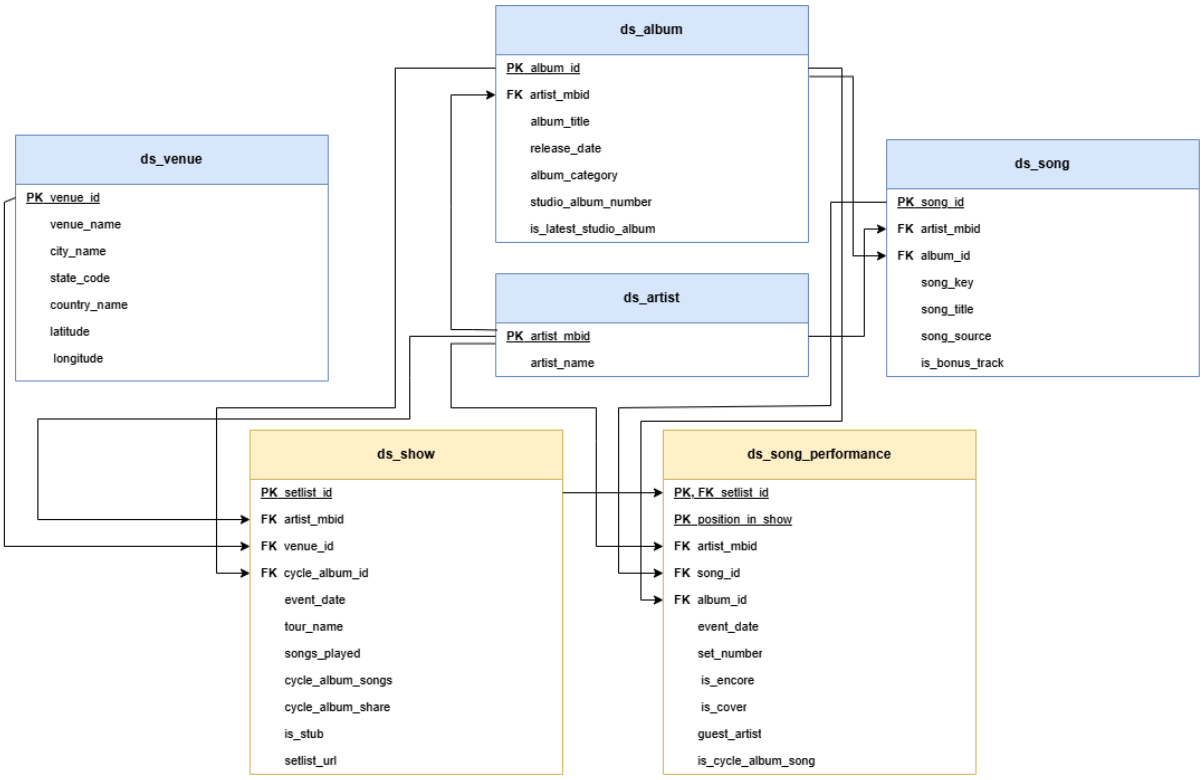
There are 6 tables: four dimensions then two facts at different grains

Table	Type	Grain	Primary key	Key fields	Description
`DS_ARTIST`	Dimension	One row per artist	`ARTIST_MBID`	`ARTIST_NAME`	The 5 artists
`DS_VENUE`	Dimension	One row per venue	`VENUE_ID`	`VENUE_NAME`, `CITY_NAME`, `STATE_CODE`, `COUNTRY_NAME`, `LATITUDE`, `LONGITUDE`	Every venue played, parsed from setlist venue JSON; feeds maps.

DS_ALBUM	Dimension	One row per core album	ALBUM_ID (MusicBrainz release group id)	ARTIST_MBID, ALBUM_TITLE, RELEASE_DATE, ALBUM_CATEGORY, STUDIO_ALBUM_NUMBER, IS_LATEST_STUDIO_ALBUM	29 curated albums: studio albums and two Kendrick Lamar non-studio releases that were relevant
DS_SONG	Dimension	One row per song per artist	SONG_ID (ARTIST_MBID + | + SONG_KEY)	SONG_KEY, SONG_TITLE, ALBUM_ID, SONG_SOURCE, IS_BONUS_TRACK	Every album song plus every song played live that is not on a core album (Cover or Non-album).
DS_SHOW	Fact	One row per past show	SETLIST_ID	ARTIST_MBID, VENUE_ID, EVENT_DATE, TOUR_NAME, CYCLE_ALBUM_ID, SONGS_PLAYED, CYCLE_ALBUM_SONGS, CYCLE_ALBUM_SHARE, IS_STUB, SETLIST_URL	Executive view: shows, markets and new-album share.

DS_SONG_PERFORMANCE	Fact	One row per song played per show	SETLIST_ID + POSITION_IN_SHOW	SONG_ID, ALBUM_ID, ARTIST_MBID, EVENT_DATE, SET_NUMBER, IS_ENCORE, IS_COVER, GUEST_ARTIST, IS_CYCLE_ALBUM_SONG	Analyst view: song retention, setlist order, row-level detail.

Schema



→ represents 1:N relationship

How the sources join

Every join is an exact id except song matching, which uses a normalized title and matches 75–85% of songs played per artist.

From	To	Join on	Cardinality	Notes
DS_SHOW	DS_ARTIST	ARTIST_MBID	Many to one	setlist.fm stores the MusicBrainz artist id
DS_SHOW	DS_VENUE	VENUE_ID	Many to one	setlist.fm venue id
DS_SHOW	DS_ALBUM	CYCLE_ALBUM_ID = ALBUM_ID	Many to one	The album the show was promoting
DS_SONG_PERFORMANCE	DS_SHOW	SETLIST_ID	Many to one	Songs within a show
DS_SONG_PERFORMANCE	DS_SONG	SONG_ID	Many to one	Built from ARTIST_MBID + normalized song title
DS_SONG_PERFORMANCE	DS_ALBUM	ALBUM_ID	Many to one	Null for covers and non-album songs
DS_SONG_PERFORMANCE	DS_ARTIST	ARTIST_MBID	Many to one	
DS_SONG	DS_ALBUM	ALBUM_ID	Many to one	Each song's respective album
DS_SONG / DS_ALBUM	DS_ARTIST	ARTIST_MBID	Many to one	

Bridging between setlist.fm and MusicBrainz: setlist.fm song titles meet MusicBrainz track titles through `SONG_KEY`: lowercase, bracketed suffixes such as (feat. …) or [Live] removed, & turned into "and", punctuation stripped. Snowflake records the primary and foreign keys but does not enforce them; Tableau relationships use the same keys.

Business rules

Rule	Definition	Why
Cycle album	The artist's latest studio album released on or before the show date	New-album share follows the calendar: Sabrina Carpenter's shows after Aug 29, 2025 count against *Man's Best Friend*
Core albums	29 hand-picked release groups; excludes bootlegs, demos, live albums, fan compilations and Mac DeMarco's instrumental *Five Easy Hot Dogs*	MusicBrainz lists 104 "albums", many unofficial
Home album	A song belongs to the earliest core album it appears on	Reissues and compilations would otherwise double-count it
Bonus track	On fewer than half of an album's editions	Separates deluxe-only songs
Tape songs	Excluded from songs played	Intros and walk-on music which are not performed
Stub shows	`IS_STUB` = no songs logged; kept in show counts	The show happened; fans did not log it
Future shows	Excluded from `DS_SHOW`	Announced but not yet played

Tool choices

Step	Tool	Why
API calls and throttling	Alteryx	Download, Generate Rows and Throttle handle paging and limits visually and the api key stays in a local file
Tracklists to one row per track	Alteryx	Nested tracklist JSON exceeded file cell limits, so it was flattened a few times before exporting out
Load	Snowflake	Three raw CSVs loaded into Snowflake
Setlist songs to rows	SQL	Unnests sets and songs while keeping their order through the flatten index
Cleaning and modeling	SQL	Date parsing, title normalization, edition collapsing, cycle-album logic in one script